

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph dan Tree

Oleh:

Amalia Soraya

NIM. 2510817120002

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI
2026**

LEMBAR PENGESAHAN

LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA

MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6 : Graph dan Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Amalia Soraya
NIM : 2510817120002

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S,Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iv
DAFTAR TABEL	v
SOAL 1 Konversi Adjacency List ke Adjacency Matrix	1
A. Source Code.....	3
B. Output Program	5
C. Pembahasan	5
SOAL 2 Konversi Adjacency Matrix ke Adjacency List.....	7
A. Source Code.....	8
B. Output Program	10
C. Pembahasan	10
SOAL 3 BFS: Jarak Terpendek Keluar Labirin	11
A. Source Code.....	13
B. Output Program	14
C. Pembahasan	15
SOAL 4 DFS: Menghitung Semua Jalur Keluar Labirin	16
A. Source Code.....	18
B. Output Program	19
C. Pembahasan	19
SOAL 5 Tree Traversal	21
A. Source Code.....	22
B. Output Program	25
C. Pembahasan	25

TAUTAN GIT	27
------------------	----

DAFTAR GAMBAR

Gambar 1. Screenshot Output Soal 1.....	5
Gambar 2. Screenshot Output Soal 2.....	10
Gambar 3. Screenshot Output Soal 3.....	14
Gambar 4. Screenshot Output Soal 4.....	19
Gambar 5. Screenshot Output Soal 5.....	25

DAFTAR TABEL

Tabel 1. Source Code file soal_1.cpp	3
Tabel 2. Source Code File soal_2.cpp	8
Tabel 3. Source Code File soal_3.cpp	13
Tabel 4. Source Code File soal_4.cpp	18
Tabel 5. Source Code File soal_5.cpp	22

SOAL 1

Konversi Adjacency List ke Adjacency Matrix

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang terdiri dari N vertex dan M edge. Setiap vertex memiliki label berupa satu karakter unik. Graph diberikan dalam bentuk daftar edge. Setiap edge ditulis sebagai U,V,W , yang berarti terdapat edge berarah dari vertex U menuju vertex V dengan bobot W . Karena graph bersifat berarah, edge U,V,W tidak otomatis berarti terdapat edge dari V menuju U . Tugas Anda adalah mengubah representasi graph tersebut menjadi adjacency matrix. Jika terdapat edge dari vertex ke- i menuju vertex ke- j , maka nilai matrix pada baris i dan kolom j adalah bobot edge tersebut. Jika tidak terdapat edge, nilainya 0.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

M

$U_1 V_1 W_1$

$U_2 V_2 W_2$

...

$U_M V_M W_M$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- M adalah jumlah edge.
- U_i dan V_i adalah vertex asal dan vertex tujuan pada edge ke- i .
- W_i adalah bobot edge ke- i .

Format Output

Cetak adjacency matrix dengan format berikut.

Adjacency Matrix:

V1 V2 ... VN

V1 a11 a12 ... a1N

V2 a21 a22 ... a2N

...

VN aN1 aN2 ... aNN

Nilai a_{ij} adalah bobot edge dari vertex V_i menuju vertex V_j . Jika edge tersebut tidak ada, nilai

a_{ij} adalah 0.

Batasan

- $2 \leq N \leq 10$
- $0 \leq M \leq N * (N - 1)$
- $1 \leq W_i \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Graph tidak memiliki self-loop.
- Graph tidak memiliki edge duplikat dengan arah yang sama.
- Graph bersifat berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 6 A B 4 A C 2 B D 7 C B 1 C E 6 D C 3	Adjacency Matrix: A B C D E A 0 4 2 0 0 B 0 0 0 7 0 C 0 1 0 0 6 D 0 0 3 0 0 E 0 0 0 0 0

Penjelasan Contoh

Edge A B 4 menghasilkan nilai 4 pada baris A kolom B. Karena graph berarah, nilai pada baris B kolom A tetap 0 selama tidak ada edge dari B menuju A. Edge D C 3 menghasilkan nilai 3 pada baris D kolom C. Vertex E tidak memiliki edge keluar, sehingga seluruh nilai pada baris E adalah 0

A. Source Code

Tabel 1. Source Code file soal_1.cpp

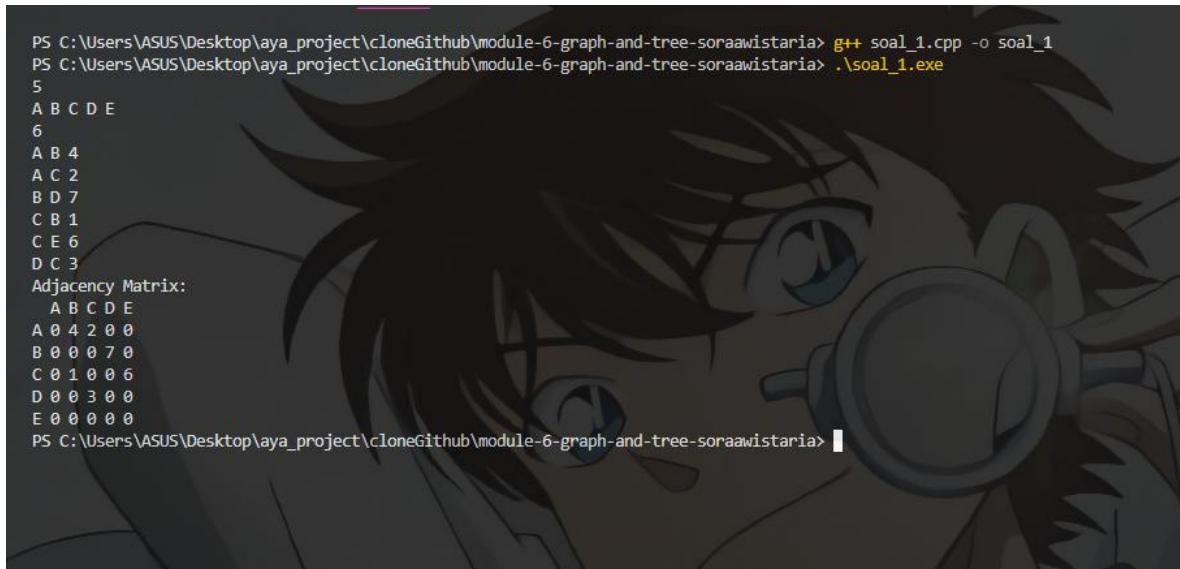
```
1  #include <iostream>
2  using namespace std;
3  #define SIZE 10
4  #define NO_EDGE -1
5
6  typedef struct {
7      int adjMatrix[SIZE][SIZE];
8      char vertex[SIZE];
9  } Graph;
10
11
12 void init(Graph *g, int n) {
13     for (int i=0; i<n; i++) {
14         for (int j=0; j<n; j++) {
15             g->adjMatrix[i][j] = NO_EDGE;
16         }
17         g->vertex[i] = 0;
18     }
19 }
20
21 void addEdge(Graph *g, int u, int v, int w, int n) {
22     if (u >= 0 && u < n && v >= 0 && v < n) {
23         g->adjMatrix[u][v] = w;
24     }
25 }
26
27
28 void addVertex (Graph *g, int v, char namaVertex, int n)
29 {
30     if (v >= 0 && v < n) {
31         g->vertex[v] = namaVertex;
32     }
33 }
34 void printGraph (Graph *g, int n) {
```

```

35     cout << "Adjacency Matrix:\n";
36
37     cout << " ";
38     for (int i = 0; i < n; i++) {
39         cout << g->vertex[i];
40         if (i < n - 1) cout << " ";
41     }
42     cout << endl;
43
44     for (int i=0; i < n; i++) {
45         cout << g->vertex[i];
46         for (int j=0; j<n; j++) {
47             cout << " ";
48             if (g->adjMatrix[i][j] == NO_EDGE) {
49                 cout << '0';
50             } else {
51                 cout << g->adjMatrix[i][j];
52             }
53         }
54         cout << endl;
55     }
56 }
57
58
59 int main() {
60     Graph g;
61     int N, M;
62     cin >> N;
63     init(&g, N);
64
65
66     for (int i=0; i<N; i++) {
67         char vertex;
68         cin >> vertex;
69         addVertex(&g, i, vertex, N);
70     }
71
72     cin >> M;
73     char U, V;
74     int W;
75     for (int i=0; i<M; i++) {
76         cin >> U >> V >> W;
77         addEdge(&g, (U - 'A'), (V - 'A'), W, N);
78     }
79
80     printGraph(&g, N);
81 }

```

B. Output Program



```
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ soal_1.cpp -o soal_1
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_1.exe
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria>
```

Gambar 1. Screenshot Output Soal 1

C. Pembahasan

Kode berikut merupakan algoritma mengubah adjacency list menjadi adjacency matrix dengan graph. Jadi, inputnya berupa nama jumlah vertex, nama vertex, jumlah edge dan weight dari edge tersebut. Nantinya outputnya berupa graph yang sudah ada weighted edge. Disini ada 1 struct dan 4 fungsi selain fungsi main.

Pertama, struct `graph`, isinya variabel array adjacency matrix dengan ukuran maksimal 10x10 sesuai constraint, lalu variabel array vertex yang menyimpan nama-nama vertex yang diwakili abjad A-Z.

Kedua, fungsi `init graph`, untuk inisialisasi graph. Cara kerjanya dengan membuat array 2D seukuran $N \times N$, semuanya diisi -1 karna belum ada edge. Lalu setiap iterasi i , vertex diisi 0 karna nama vertexnya belum diinput. Intinya ini menyiapkan container. Time complexity-nya jadi $O(n^2)$ karna iterasi i dan j sampai N keduanya, space complexity juga $O(n^2)$ karena ini membuat array dengan ukuran $N \times N$ dan array vertex ukuran N . Jadi kalau N nya 5, perlu array ukuran 25 buat menyimpan graph dan weighted edge nya. Perlu array ukuran 5 juga buat nyimpan char vertex.

Ketiga, fungsi `addEdge`, buat tambah edge. Cara kerjanya dengan mengisi nilai w di indeks u ke v , kalau $0 \leq u < n$ dan $0 \leq v < n$. Jadi ya parameter fungsinya ada `struct graph`, u (vertex awal), v (vertex tujuan), w (weight edge), dan n (ukuran matrix). Time complexity-nya $O(1)$ karena dia langsung memasukkan edge ke graph yang sudah di inisialisasi, space complexity-nya $O(1)$ karena dia cuma tambah w ke graph yang sudah ada.

Keempat, fungsi `addVertex`, buat tambah vertex. Cara kerjanya dengan menambah char nama vertex hasil input ke array vertex yang sudah diinit sebelumnya. Dia tambah nama vertex kalau $0 \leq v < n$ agar tidak out of index. Time complexity-nya $O(1)$ karna dia langsung tambah nama vertex ke array yang sudah ada (tanpa loop lagi) dan space complexity-nya $O(1)$ karna dia cuma tambah char ke array yang sudah ada.

Kelima, fungsi `printGraph` buat mencetak graph yang sudah ada vertex, nama, dan edge nya. Cara kerjanya, baris pertama itu print kata adjacency matrix dulu, lalu baris kedua itu print nama-nama vertex, lalu baru print matrix ukuran $N \times N$ tadi. Time complexity-nya $O(N^2)$, $N + N^2$, pertama buat print nama vertex, lalu $N \times N$ buat print adjacency matrix.

Terakhir, fungsi `main`, fungsi utama tempat input dan output serta proses program. Pertama-tama ada deklarasi `graph`, N , M . Lalu input N . Lalu inisialisasi `graph`. Lalu loop 0 sampai N untuk input vertex. Lalu input M dan deklarasi variabel U , V , W . Lalu loop 0 sampai M (jumlah edge) untuk menambah edge ke graph. Terakhir memanggil fungsi `printGraph`. Time complexity-nya $O(N + M + N^2)$, karena loop input vertex, input edge-edge untuk graph, dan cetak output graph yang sampai N^2 .

SOAL 2

Konversi Adjacency Matrix ke Adjacency List

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah directed weighted graph yang direpresentasikan dalam bentuk adjacency matrix berukuran $N \times N$. Setiap vertex memiliki label berupa satu karakter unik. Nilai matrix pada baris i dan kolom j menunjukkan bobot edge dari vertex ke- i menuju vertex ke- j . Jika nilainya 0, berarti tidak terdapat edge dari vertex ke- i menuju vertex ke- j . Tugas Anda adalah mengubah adjacency matrix tersebut menjadi adjacency list. Untuk setiap vertex, cetak semua vertex tujuan yang dapat dicapai langsung beserta bobot edge-nya. Urutan vertex tujuan mengikuti urutan label pada input.

Format Input

Input diberikan dalam format berikut.

N

$V_1 V_2 \dots V_N$

$A_{11} A_{12} \dots A_{1N}$

$A_{21} A_{22} \dots A_{2N}$

...

$A_{N1} A_{N2} \dots A_{NN}$

Keterangan:

- N adalah jumlah vertex.
- V_i adalah label vertex ke- i .
- A_{ij} adalah nilai matrix pada baris ke- i dan kolom ke- j .
- Jika $A_{ij} > 0$, maka terdapat edge dari V_i menuju V_j dengan bobot A_{ij} .
- Jika $A_{ij} = 0$, maka tidak terdapat edge dari V_i menuju V_j .

Format Output

Cetak adjacency list untuk setiap vertex dengan format berikut.

Adjacency List:

$V_1 \rightarrow (X_1, w_1), (X_2, w_2), \dots$

V2 -> (X1,w1), (X2,w2), ...

...

VN -> ...

Jika sebuah vertex tidak memiliki edge keluar, cetak tanda - setelah simbol ->.

Constraints

- $2 \leq N \leq 10$
- $0 \leq A_{ij} \leq 1000$
- Label vertex berupa karakter alfabet kapital A sampai Z.
- Semua label vertex dijamin unik.
- Nilai diagonal utama matrix dijamin 0.
- Nilai 0 berarti tidak ada edge.
- Nilai positif berarti bobot edge.
- Matrix merepresentasikan graph berarah dan berbobot.

Contoh Kasus

Input	Output
5 A B C D E 0 4 2 0 0 0 0 0 7 0 0 1 0 0 6 0 0 3 0 0 0 0 0 0 0	Adjacency List: A -> (B,4), (C,2) B -> (D,7) C -> (B,1), (E,6) D -> (C,3) E -> -

Penjelasan Contoh

Pada baris vertex A, nilai positif muncul pada kolom B dan C. Karena itu, adjacency list untuk A adalah (B,4), (C,2).

Pada baris vertex E, semua nilai bernilai 0, sehingga vertex E tidak memiliki edge keluar dan output-nya adalah E -> -.

A. Source Code

Tabel 2. Source Code File soal_2.cpp

1	#include <iostream>
2	#include <vector>
3	using namespace std;
4	
5	int main() {

```

6      int N;
7      cin >> N;
8
9      char vertex[N];
10     for (int i=0; i<N; i++) {
11         cin >> vertex[i];
12     }
13
14     int Graph[N][N];
15     for (int i=0; i<N; i++) {
16         for (int j=0; j<N; j++) {
17             cin >> Graph[i][j];
18         }
19     }
20
21     cout << "\nAdjacency List : \n";
22     for (int i=0; i<N; i++) {
23         cout << vertex[i] << " -> ";
24
25         bool hasEdge = false;
26         for (int j=0; j<N; j++) {
27             if (Graph[i][j] > 0) {
28                 if (hasEdge) cout << ", ";
29                 cout << "(" << vertex[j] << ", " <<
Graph[i][j] << ")";
30                 hasEdge = true;
31             }
32         }
33
34         if (!hasEdge) cout << "-";
35
36         cout << endl;
37     }
38 }

```

B. Output Program



```
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ soal_2.cpp -o soal_2
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_2.exe
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0

Adjacency List :
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> |
```

Gambar 2. Screenshot Output Soal 2

C. Pembahasan

Kode berikut merupakan algoritma mengubah adjacency matrix menjadi adjacency list. Inputnya berupa N yaitu jumlah vertex dan adjacency matrix yang sudah berisi weighted edge. Outputnya adjacency list yang menyebutkan edge-edge dari vertex ini ke vertex itu.

Masuk ke fungsi main, cara kerjanya pertama-tama, menerima input nilai N dan nama-nama vertex ke array vertex. Lalu input adjacency matrix dengan ukuran $N \times N$. Lalu loop i dari 0 sampai N untuk print adjacency list setiap vertex. Setiap baris, print nama vertex dan tanda panah. Lalu loop j dari 0 sampai N untuk edge setiap kolom. Kalau edge lebih dari 0, print nama vertex beserta weighted edge tersebut. hasEdge digunakan untuk mengeprint koma jika sebelumnya sudah ada edge yang di print. Jika vertex tidak memiliki edge, print strip (-).

Time complexity $O(n^2)$ karena input adjacency matrix dan print adjacency list. Space complexity $O(n^2)$ karena menyimpan vertex sebanyak N dan menyimpan adjacency matrix dari input.

SOAL 3

BFS: Jarak Terpendek Keluar Labirin

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Sebuah labirin direpresentasikan sebagai grid berukuran $R \times C$. Setiap sel pada grid memiliki nilai sebagai berikut.

- 0 menyatakan jalan yang dapat dilewati.
- 1 menyatakan dinding yang tidak dapat dilewati.

Diberikan posisi awal dan posisi tujuan. Anda harus menentukan jumlah langkah minimum dari posisi awal menuju posisi tujuan menggunakan algoritma Breadth-First Search (BFS). Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan. Pergerakan diagonal tidak diperbolehkan. Beberapa jalur pada labirin dapat berupa jalan buntu, sehingga algoritma harus tetap memproses kemungkinan jalur secara sistematis.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

X

Keterangan:

- X adalah jumlah langkah minimum dari posisi awal ke posisi tujuan.
- Jika posisi tujuan tidak dapat dicapai, cetak -1 .

Constraints

- $1 \leq R \leq 100$
- $1 \leq C \leq 100$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Algoritma yang digunakan wajib BFS.

Contoh Kasus

Input	Output
8 10 0 0 0 0 1 0 0 0 0 0 1 1 1 0 1 0 1 1 1 0 0 0 1 0 0 0 0 0 1 0 0 1 1 1 1 1 1 0 1 0 0 0 0 0 0 0 1 0 0 0 1 1 1 1 1 0 1 1 1 0 0 0 0 0 1 0 0 0 1 0 0 1 1 0 0 0 1 0 0 0 0 0 7 9	16

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (7,9). Labirin memiliki beberapa percabangan dan jalan buntu, misalnya cabang menuju area kiri bawah dan cabang menuju sel (2,1).

Dengan BFS, setiap sel diproses berdasarkan jarak terdekat dari start. Jarak minimum dari (0,0) ke (7,9) adalah 16 langkah, sehingga output yang dicetak hanya 16.

A. Source Code

Tabel 3. Source Code File soal_3.cpp

1	#include <iostream>
2	#include <queue>
3	using namespace std;
4	
5	struct Node {
6	int r, c, jarak;
7	};
8	
9	int dr[] = {-1, 1, 0, 0};
10	int dc[] = {0, 0, -1, 1};
11	
12	int main() {
13	int R, C, x_awal, y_awal, x_tuju, y_tuju;
14	cin >> R >> C;
15	
16	int G[R][C];
17	for (int i=0; i<R; i++) {
18	for (int j=0; j<C; j++) {
19	cin >> G[i][j];
20	}
21	}
22	cin >> x_awal >> y_awal;
23	cin >> x_tuju >> y_tuju;
24	
25	bool visited[R][C];
26	for (int i=0; i < R; i++) {
27	for (int j=0; j < C; j++) {
28	visited[i][j] = false;
29	}
30	}
31	
32	queue<Node> q;
33	
34	q.push({x_awal, y_awal, 0});
35	visited[x_awal][y_awal] = true;
36	
37	int totalLangkah = -1;
38	
39	while (!q.empty()) {
40	Node curr = q.front();

```

41         q.pop();
42
43         if (curr.r == x_tuju && curr.c == y_tuju) {
44             totalLangkah = curr.jarak;
45             break;
46         }
47
48         for (int i=0; i < 4; i++) {
49             int nr = curr.r + dr[i];
50             int nc = curr.c + dc[i];
51
52             if (nr >= 0 && nr < R && nc >= 0 && nc < C)
53             {
54                 if (G[nr][nc] == 0 && !visited[nr][nc])
55                 {
56                     visited[nr][nc] = true;
57                     q.push({nr, nc, curr.jarak + 1});
58                 }
59             }
60
61             cout << totalLangkah;
62     }

```

B. Output Program



```

PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ soal_3.cpp -o soal_3
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_3.exe
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria>

```

Gambar 3. Screenshot Output Soal 3

C. Pembahasan

Kode berikut merupakan algoritma BFS (Breadth-First Search), bekerja dengan visit tetangga (titik terdekat, mau itu atas bawah kiri kanan) sebelum melangkah lebih jauh. Gunanya untuk mencari jarak paling pendek untuk mencapai titik tujuan.

Pertama, ada struct `Node`, gunanya menyimpan posisi titik sekarang. Isinya `r` dan `c`, titik koordinat di baris sekian dan kolom sekian, serta jarak (jumlah langkah dari posisi awal hingga posisi ini). Lalu ada array global `dr` dan `dc`, distance row dan distance column. Menyimpan angka untuk perhitungan arah gerak, bisa ke atas, kanan, bawah, kiri.

Lalu masuk ke fungsi `main`. Cara kerjanya pertama input `R`, `C`, lalu graph `RxC` dan koordinat awal dan tujuan. Lalu inisialisasi array `visited` seukuran `RxC` dan semuanya diisi `false`. Lalu inisialisasi queue BFS, lalu push posisi awal `(0, 0)` dan jarak `0`. Lalu tandai itu sebagai `visited = true`. `totalLangkah` diisi `-1` kalau tujuan tidak bisa dicapai. Lalu loop BFS selama queue tidak kosong, ambil node paling depan, kalau koordinatnya sama dengan tujuan, simpan jaraknya ke `totalLangkah` dan break loop. Kalau belum sampai tujuan, loop `i 0` sampai `3` untuk mengecek titik yang cocok untuk dilewati. Misalnya node berada di `(0, 0)`, maka ia mengecek ke atas, kiri, kanan, bawah dahulu. Kalau titik itu ada di dalam grid, belum visited, dan nilainya `0` (bukan tembok), maka melangkah ke titik itu dan titik itu ditandai `visited = true`. Lalu di push ke queue dan jarak bertambah `1`. Setelah BFS selesai, print total langkah yang dilewati.

Time complexity $O(RxC)$, $((R \times C) \times 4)$ karena ada loop untuk membuat graph berukuran `RxC` dan while loop untuk jalan yang di dalamnya ada loop untuk menjelajahi titik juga, space complexity $O(RxC)$ karena perlu array 2D untuk graph itu.

SOAL 4

DFS: Menghitung Semua Jalur Keluar Labirin

Time Limit: 2.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan sebuah labirin berukuran $R \times C$ dengan aturan sel yang sama seperti soal sebelumnya. Anda harus menghitung banyaknya jalur sederhana dari posisi awal menuju posisi tujuan menggunakan algoritma Depth-First Search (DFS) rekursif. Sebuah jalur disebut sederhana jika tidak mengunjungi sel yang sama lebih dari satu kali dalam jalur yang sama. Dari sebuah sel, Anda hanya boleh bergerak ke empat arah: atas, bawah, kiri, dan kanan.

Labirin dapat memiliki beberapa percabangan dan jalan buntu. Karena itu, DFS harus melakukan proses backtracking agar dapat mencoba semua kemungkinan jalur yang valid.

Format Input

Input diberikan dalam format berikut.

R C

G_{11} G_{12} ... G_{1C}

G_{21} G_{22} ... G_{2C}

...

G_{R1} G_{R2} ... G_{RC}

SR SC

FR FC

Keterangan:

- R adalah jumlah baris grid.
- C adalah jumlah kolom grid.
- G_{ij} adalah nilai sel pada baris ke- i dan kolom ke- j .
- (SR, SC) adalah posisi awal.
- (FR, FC) adalah posisi tujuan.

Indeks baris dan kolom dimulai dari 0.

Format Output

Cetak satu bilangan bulat.

T

Keterangan:

- T adalah jumlah jalur sederhana dari posisi awal ke posisi tujuan.
- Jika tidak ada jalur yang dapat dilalui, cetak 0.

Constraints

- $1 \leq R \leq 7$
- $1 \leq C \leq 7$
- G_{ij} hanya bernilai 0 atau 1.
- $0 \leq SR < R$
- $0 \leq SC < C$
- $0 \leq FR < R$
- $0 \leq FC < C$
- Sel start dan finish dijamin bernilai 0.
- Pergerakan hanya boleh dilakukan ke sel bernilai 0.
- Satu jalur tidak boleh mengunjungi sel yang sama lebih dari satu kali.
- Jumlah jalur valid pada test case dijamin tidak lebih dari 100000.
- Algoritma yang digunakan wajib DFS rekursif dengan backtracking.

Contoh Kasus

Input	Output
5 6 0 0 0 1 0 0 0 1 0 0 0 1 0 1 0 1 0 0 0 0 0 1 1 0 1 1 0 0 0 0 0 0 4 5	4

Penjelasan Contoh

Posisi awal berada pada (0,0) dan posisi tujuan berada pada (4,5). Labirin memiliki lebih dari satu jalur valid serta beberapa cabang yang tidak menghasilkan solusi, misalnya cabang menuju sel (0,5).

Dengan DFS dan backtracking, seluruh jalur sederhana dari start ke finish dapat dihitung. Pada contoh ini terdapat 4 jalur valid, sehingga output yang dicetak hanya 4.

A. Source Code

Tabel 4. Source Code File soal_4.cpp

```

1  #include <iostream>
2  using namespace std;
3
4  int R, C, x_tuju, y_tuju;
5  int G[7][7];
6  bool visited[7][7];
7  int totalJalur = 0;
8
9  int dr[] = {-1, 1, 0, 0};
10 int dc[] = {0, 0, -1, 1};
11
12 void dfs(int r, int c) {
13     if (r == x_tuju && c == y_tuju) {
14         totalJalur++;
15         return;
16     }
17
18     for (int i=0; i<4; i++) {
19         int nr = r + dr[i];
20         int nc = c + dc[i];
21
22         if (nr >= 0 && nr < R && nc >= 0 && nc < C &&
23             G[nr][nc] == 0 && !visited[nr][nc]) {
24             visited[nr][nc] = true;
25             dfs(nr, nc);
26             visited[nr][nc] = false;
27         }
28     }
29 }
30
31 int main() {
32     cin >> R >> C;
33
34     for (int i=0; i<R; i++) {
35         for (int j=0; j<C; j++) {
36             cin >> G[i][j];
37         }
38     }
39     int x_awal, y_awal;

```

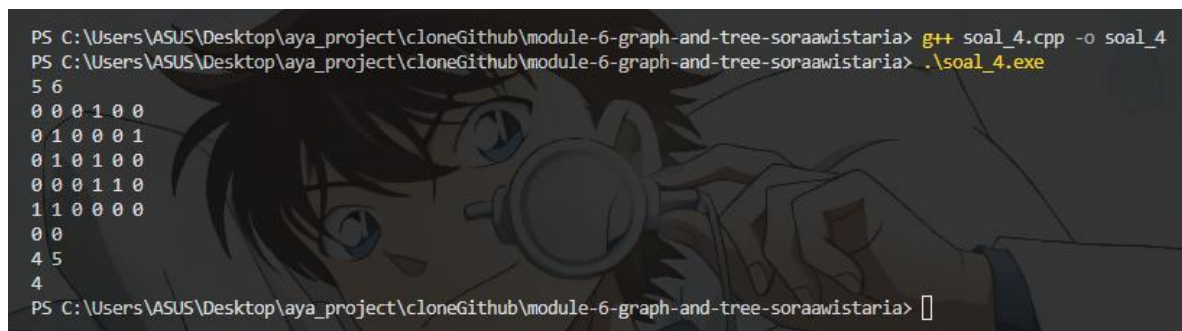


```

40     cin >> x_awal >> y_awal;
41     cin >> x_tuju >> y_tuju;
42
43     visited[x_awal][y_awal] = true;
44     dfs(x_awal, y_awal);
45
46     cout << totalJalur;
47 }

```

B. Output Program



```

PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ soal_4.cpp -o soal_4
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_4.exe
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> 

```

Gambar 4. Screenshot Output Soal 4

C. Pembahasan

Kode berikut merupakan algoritma DFS (Depth-First Search), bekerja dengan menelusuri semua sampai titik tujuan. Algoritma ini memilih 1 jalur, lalu jalani jalur itu hingga sampai tujuan atau mentok nabrak jika tujuan tidak sampai. Cara kerjanya dengan implementasi stack dan fungsi rekursif.

Pertama, ada variabel global, agar variabel ini bisa digunakan di fungsi main dan fungsi DFS. Variabel globalnya yaitu R dan C untuk ukuran grid graph, x_tuju dan y_tuju untuk koordinat tujuan, G[7][7] untuk membuat grid (ukuran 7 sesuai constraint), visited[7][7] untuk menyimpan jalur yang sudah dikunjungi, dan total jalur untuk menyimpan berapa jalur yang bisa dipakai untuk sampai tujuan. Lalu array dr dan dc untuk menyimpan hitungan 4 arah gerak, atas bawah kiri kanan.

Kedua, fungsi dfs untuk mencoba semua jalur yang mungkin, dari titik awal sampai titik tujuan. Cara kerjanya yaitu membuat base case dulu, kalau posisi sekarang sudah sama dengan titik koordinat tujuan, total jalur ditambah 1 lalu return. Kalau belum sampai tujuan, cek atas bawah kiri kanan. Titik yang valid (tidak diluar grid, nilainya 0, dan belum visited)

akan dilangkah dan ditandai `visited = true`. Lalu rekursi dari titik sekarang. Setelah rekursi selesai, titik tadi ditandai `visited = false` agar jalur bisa dipakai lagi untuk eksplorasi cabang lain. Time complexity-nya $O(4^{(RxC)})$ karena setiap titik bisa bercabang ke 4 arah, mencoba ke 4 arah tersebut sebanyak ukuran grid graph. Sedangkan space complexity-nya $O(RxC)$ karena menyimpan grid graph seukuran RxC dan call stack yang bisa mencapai RxC jika worst case (misalnya tidak ada obstacle, semua grid nilainya 0).

Terakhir, fungsi `main`. Dimulai dari input R dan C , lalu input grid graph, lalu input koordinat awal dan tujuan. Lalu posisi awal ditandai `visited = true` agar tidak balik ke koordinat awal selama proses DFS. Lalu panggil fungsi DFS dari posisi awal, lalu print total jalur. Time complexity-nya $O(RxC + 4^{(RxC)})$ untuk input grid + fungsi dfs berjalan, space complexity $O(RxC)$ karena memori untuk grid graph, titik visited, dan call stack.

SOAL 5

Tree Traversal

Time Limit: 1.0 s
Memory Limit: 64 MB

Deskripsi Soal

Diberikan N bilangan bulat. Masukkan seluruh bilangan tersebut ke dalam sebuah Red-Black Tree secara berurutan sesuai input. Jika terdapat bilangan duplikat, bilangan tersebut diabaikan dan tidak dimasukkan kembali ke tree. Setelah RBTree terbentuk, diberikan Q query. Setiap query meminta salah satu hasil traversal berikut.

- PREORDER, yaitu urutan root, subtree kiri, lalu subtree kanan.
- INORDER, yaitu urutan subtree kiri, root, lalu subtree kanan.
- POSTORDER, yaitu urutan subtree kiri, subtree kanan, lalu root.
- ALL, yaitu mencetak ketiga traversal sekaligus.

Tugas Anda adalah mencetak hasil traversal sesuai query yang diberikan.

Format Input

Input diberikan dalam format berikut.

N

$A_1 A_2 \dots A_N$

Q

T_1

T_2

...

T_Q

Keterangan:

- N adalah banyaknya bilangan yang akan dimasukkan ke RBTree.
- A_i adalah bilangan ke- i .
- Q adalah banyaknya query traversal.
- T_i adalah jenis traversal yang diminta.

Format Output

Untuk setiap query, cetak hasil traversal sesuai format berikut.

[Preorder] : daftar_angka

[Inorder] : daftar_angka

[Postorder] : daftar_angka

Jika query adalah ALL, cetak ketiga baris traversal dengan urutan Preorder, Inorder, lalu

Postorder.

Jika setelah penghapusan duplikat tree tetap kosong, cetak:

Tree kosong. Tidak ada yang bisa ditampilkan.

Constraints

- $0 \leq N \leq 1000$
- $-10000000000 \leq A_i \leq 10000000000$
- $1 \leq Q \leq 20$
- T_i hanya salah satu dari PREORDER, INORDER, POSTORDER, atau ALL.

Contoh Kasus

Input	Output
7	[Preorder] : 50 30 20 40 70 60 80
50 30 70 20 40 60 80	[Inorder] : 20 30 40 50 60 70 80
4	[Postorder] : 20 40 30 60 80 70 50
PREORDER	[Preorder] : 50 30 20 40 70 60 80
INORDER	[Inorder] : 20 30 40 50 60 70 80
POSTORDER	[Postorder] : 20 40 30 60 80 70 50
ALL	

Penjelasan Contoh

Bilangan dimasukkan ke RBTree secara berurutan. Nilai 50 menjadi root. Nilai yang lebih kecil dari 50 masuk ke subtree kiri, sedangkan nilai yang lebih besar masuk ke subtree kanan.

Hasil INORDER pada RBTree selalu menghasilkan urutan nilai dari kecil ke besar. Pada contoh ini, hasil inorder adalah 20 30 40 50 60 70 80, sehingga struktur RBTree yang dibentuk sudah konsisten dengan aturan RBTree.

A. Source Code

Tabel 5. Source Code File soal_5.cpp

1	#include "RedBlackTree.h"
---	---------------------------

```

2  #include <iostream>
3  #include <vector>
4  using namespace std;
5
6  void preorder (const RedBlackTree::Node* node, const
RedBlackTree::Node* nil, vector<int>& hasil) {
7      if (node->isNil) return;
8      hasil.push_back(node->key);
9      preorder(node->left, nil, hasil);
10     preorder(node->right, nil, hasil);
11 }
12
13 void inorder (const RedBlackTree::Node* node, const
RedBlackTree::Node* nil, vector<int>& hasil) {
14     if (node->isNil) return;
15     inorder(node->left, nil, hasil);
16     hasil.push_back(node->key);
17     inorder(node->right, nil, hasil);
18 }
19
20 void postorder(const RedBlackTree::Node* node, const
RedBlackTree::Node* nil, vector<int>& hasil) {
21     if (node->isNil) return;
22     postorder(node->left, nil, hasil);
23     postorder(node->right, nil, hasil);
24     hasil.push_back(node->key);
25 }
26
27 void print(string label, vector<int>& hasil) {
28     cout << label << " : ";
29     for (int i=0; i< hasil.size(); i++) {
30         if (i > 0) cout << " ";
31         cout << hasil[i];
32     }
33     cout << endl;
34 }
35
36 int main() {
37     RedBlackTree rbt;
38
39     int N;
40     cin >> N;
41
42     for (int i=0; i<N; i++) {
43         int num;
44         cin >> num;
45         if (!rbt.contains(num)) {

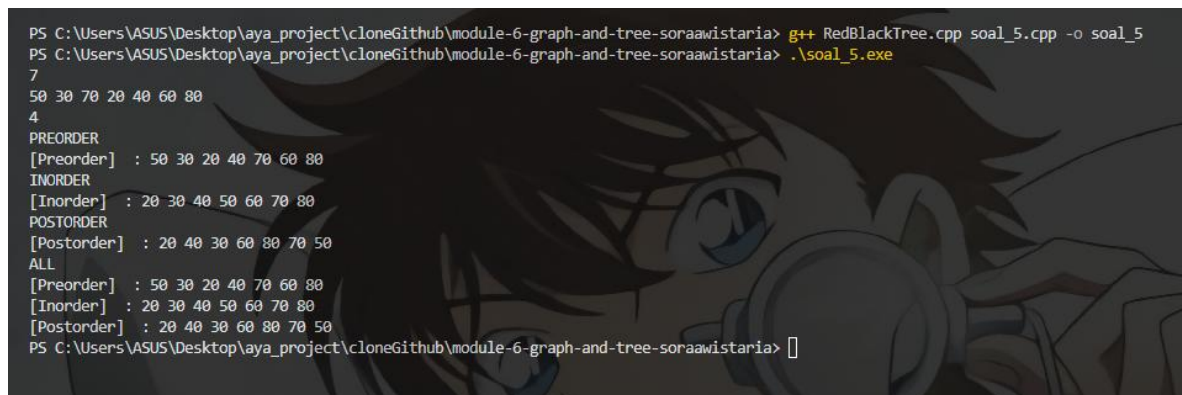
```

```

46         rbt.insert(num);
47     }
48 }
49
50 int Q;
51 cin >> Q;
52
53 for (int i=0; i<Q; i++) {
54     string query;
55     cin >> query;
56
57     if (rbt.empty()) {
58         cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
59         continue;
60     }
61
62     vector<int> pre, in, post;
63     preorder(rbt.root(), rbt.nil(), pre);
64     inorder(rbt.root(), rbt.nil(), in);
65     postorder(rbt.root(), rbt.nil(), post);
66
67     if (query == "PREORDER") {
68         print("[Preorder] ", pre);
69     } else if (query == "INORDER") {
70         print("[Inorder] ", in);
71     } else if (query == "POSTORDER") {
72         print("[Postorder] ", post);
73     } else if (query == "ALL") {
74         print("[Preorder] ", pre);
75         print("[Inorder] ", in);
76         print("[Postorder] ", post);
77     }
78 }
79 }

```

B. Output Program



```
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> g++ RedBlackTree.cpp soal_5.cpp -o soal_5
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> .\soal_5.exe
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder] : 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder] : 20 40 30 60 80 70 50
PS C:\Users\ASUS\Desktop\aya_project\cloneGithub\module-6-graph-and-tree-soraawistaria> 
```

Gambar 5. Screenshot Output Soal 5

C. Pembahasan

Kode berikut merupakan algoritma untuk traverse struktur data tree dengan preorder, inorder, maupun postorder.

Fungsi `preorder`, untuk traverse dari root ke kiri lalu ke kanan. Cara kerjanya, membuat base case jika node sekarang adalah nil node maka langsung return. Kalau bukan, simpan (push) node sekarang ke vector `hasil`. Lalu rekursif ke kiri, rekursif lagi ke kanan. Time complexity $O(n)$ karena setiap node di visit sekali, space complexity $O(n)$ karena menyimpan nilai node ke vector dan call stack untuk fungsi rekursi.

Fungsi `inorder`, untuk traverse dari kiri ke root lalu ke kanan. Cara kerjanya, membuat base case sama seperti preorder. Kalau bukan nil, rekursif ke kiri, push nilai `key` ke vector `hasil`, lalu rekursif lagi ke kanan. Time complexity $O(n)$ karena setiap node di visit sekali, space complexity $O(n)$ karena menyimpan nilai node ke vector dan call stack untuk fungsi rekursi.

Fungsi `postorder`, untuk traverse kiri ke kanan lalu ke root. Cara kerjanya, membuat base case sama seperti preorder. Kalau bukan nil, rekursif kiri lalu rekursif kanan. Lalu push `key` ke vector `hasil`. Time complexity $O(n)$ karena setiap node di visit sekali, space complexity $O(n)$ karena menyimpan nilai node ke vector dan call stack untuk fungsi rekursi. Fungsi `print`, untuk print hasil traverse masing-masing jenis traversal. Cara kerjanya, print label alias jenis order, lalu loop vector `hasil` dan print setiap elemennya. Time complexity

untuk mengeprint isi vector $O(n)$ sedangkan space complexity $O(1)$ karena tidak menggunakan memori baru.

Fungsi `main`, cara kerjanya inisialisasi `RedBlackTree` dahulu. Lalu menerima input `N` dan setiap angka untuk dibuat tree. Setiap angka di cek, agar tidak ada angka yang duplikat. Kalau tidak duplikat, `insert` ke tree. Lalu input `Q` untuk berapa kali query dan querynya apa. Setiap iterasi di cek dulu apakah tree kosong atau tidak. Kalau kosong, print pesan dan `continue` ke query berikutnya. Kalau tree tidak kosong, traversal tree dengan ketiga jenis traverse-nya. Hasil traverse di print sesuai query, mau itu preorder, inorder, postorder, maupun all (ketiganya). Time complexity-nya $O(n \log n + Q \times N)$ karena insert elemen ke RBT $O(\log n)$ dan query traversal $O(n)$. Space complexity $O(n)$ karena menyimpan tree dan vector traversal.

TAUTAN GIT

Repository : <https://github.com/DSA26-ULM/module-6-graph-and-tree-soraawistaria>